represented using an array of `N` elements, without pointers to children or parents. Figure 2.4 shows how the tree and array representation of a heap are equivalent.

In the array representation, if we are starting to count indices from `0`, the children of the `i`-th node are stored in the positions[8] `(2 * i) + 1` and `2 * (i + 1)`, while the parent of node `i` is at index `(i - 1) / 2` (except for the root, which has no parent). For example, looking at figure 2.4, the node at index `1` (whose priority is `3`) has two children at indices `3` and `4`, and its parent is at index `0`; if we consider a node at position `5`, its parent is the element at index `2` and its children (not shown in the picture) would be at indices `11` and `12`.

It might seem counter-intuitive that we use an array to represent a tree. After all, trees were invented to overcome arrays limitations. This is generally true, and trees have a number of advantages: they are more flexible and, if balanced, allow for better performance, with worst-case logarithmic search, insertion and delete.